

向量化处理与向量数据库

向量化：句子级 vs 词元级 Token

词元 Token 级向量化

方法：模型处理单个词或子词（如 BERT 的 WordPiece）。

输入：句子被拆分为多个 token（如 ["The", "Apollo", "program", ...]）。

输出：每个 token 生成 1 个向量（形状为 (seq_length, d_model)）。

特点：细粒度语义，但需额外处理（如池化）才能用于检索。

句子级向量化

方法：使用预训练模型（如 all-MiniLM-L6-v2）对整个句子进行编码。

输入：完整句子（如 "The Apollo program..."）。

输出：每个句子生成 1 个 固定长度的向量（如 384 维）。

特点：保留句子整体语义，适合段落检索。

RAG 中通常使用词元级向量化，因检索目标是段落或文档块。句子向量化（如 all-MiniLM-L6-v2）和 token 向量化（如 BERT 的 WordPiece）的区别是：句子向量化是先把每一个 token 计算得到一个向量，然后池化得到整个句子的向量，这也是为什么句子级向量化加载模型的时候会同步加载一个池化模型，具体步骤为：

对每个句子进行分词 → 生成 token 向量 → 通过池化（平均或 CLS）合并为句子向量。

FAISS 的本质-向量和索引

向量库的本质是将文本、提取的图片音视频内容等转换成的高维向量按照一定的索引结构存储到 index 索引文件，类似于原来的文本存储到 docx, xls, pdf 等类型文件里。索引结构通过分块，量化，图结构提供了高效的近似性检索以及存储能力，不是匹配文本而是匹配向量相似度。因此向量库就是一个高效的基于向量相似度的搜索引擎，向量库本身可以理解为文档内容的另一种存储形式。

FAISS 是最常用的一个向量索引库，但是仅负责“高效相似度搜索”，无持久化存储、元数据管理、网络接口、分布式等功能，需嵌入程序本地调用。

那么 faiss 等向量库到底是如何存储很多向量的呢？或者说这些向量存储到哪里了呢？

先来看一下这些向量自身的数据结构，这些向量本身是一个二维数组，每一行代表一个句子的向量，行数代表句子的数量。每一行的行号就是向量的逻辑 id（从 0 开始），经嵌入模型生成后，无论向量物理结构和存储地址如何变化，向量的逻辑 id 就不再变化，

而为了提高检索能力，向量库提供特殊的数据结构——索引（现在可简单类比列表或字典等，后面会详细介绍），来存储一行或多行向量。存储分为 2 个步骤，一是在训练和创建过程中，会把向量存储到索引结构里，二是会把索引结构

写入到索引文件里（类比把文本写入到 word 里）。索引会有一个属性 **ntotal** 记录**当前索引**中存储的向量总数；

因为 faiss 向量库存储检索返回的都是向量，但在 RAG 等实际应用中，需要对应的原始数据，因此，向量库需要设置一个**额外的存储结构**，用以存储高维向量和原始数据的对应关系。在基本使用时，需要手动维护一个字典，保存向量逻辑 id(键)和原始数据(值)的对应关系，在 langchain 框架下，自带了一个 docstore 自动维护。

FAISS 构建向量库的基本步骤

第一步 文本向量化

加载嵌入模型

model = SentenceTransformer('all-MiniLM-L6-v2') # 句子级编码模型

生成向量并获取向量维度

embeddings = model.encode(texts) # 输入 4 个句子，输出形状 (4, 384)

dimension = embeddings.shape[1] # 384，这一步是获取向量的维度，因为存储时需要根据维度大小分配空间，若向量维度不符合，则报错。

第二步 创建 id 和文本的映射关系字典 **docstore**

docstore = {} # 文档存储：向量 ID -> 原始内容

#原始文本存入 **docstore**

for id,text in enumerate(texts):

docstore[id] = text

第三步 索引添加向量

创建索引

index = faiss.IndexFlatL2(dimension) # 初始化索引结构，按照顺序赋予 0,1,2 标签

#索引添加向量

index.add(embeddings) # 将向量添加到索引

在添加数据前后，索引对象的内存地址不会变化，**索引对象本身是同一个**：add 操作只是将向量数据添加到这个预先创建好的索引结构中，并没有创建一个新的索引对象。

#保存索引

faiss.write_index(index, "knowledge_base.index") # 其中 **index** 是待写的索引内容，**knowledge_base.index** 是待要写入的**索引文件路径名称**（类比 docx 等文档内容写入）

#加载索引

index = faiss.read_index("index_file.index") #参数为文件路径，类比 docx 等文档内容读取

#检索问题向量化

query_vector = embed("问题文本")

#计算距离，返回相似向量 ids

```
distances, ids = index.search(query_vector, k=5)
```

根据搜索得到的 id 返回对应的文本：

```
results = [docstore[id] for id in ids[0]] #通过 docstore 找回原始文本
```

FAISS 索引结构

IndexFlatL2

是一个连续数组，存储所有的原始向量数据（可以类比列表），因此索引 id 隐式从 0 开始按照顺序增加，对应原始向量的行下标。

输入数据

```
vectors = [  
    [0.1, 0.2, 0.3, 0.4], # ID=0  
    [0.5, 0.6, 0.7, 0.8], # ID=1  
    [0.9, 1.0, 1.1, 1.2] # ID=2  
]
```

IndexFlatL2 索引

内存中的二维数组 (float32)

```
index = [  
    [0.1, 0.2, 0.3, 0.4],  
    [0.5, 0.6, 0.7, 0.8],  
    [0.9, 1.0, 1.1, 1.2]]
```

使用 **reconstruct()** 方法输出特定位置的向量

语法: **reconstruct(vector_id)**

vector_id_to_fetch = 2 # 我们想获取索引中第 3 个向量（索引从 0 开始）

```
reconstructed_vector = index.reconstruct(vector_id_to_fetch)
```

IndexFlatL2 这种索引结构，在向量搜索时采用**暴力搜索**，会计算查询向量和所有向量的相似度，时间复杂度较高，速度慢，内存占用大，用于**精确搜索**。适合**小规模数据集**或作为其他索引的量化器。

改进方向从**聚类**和**图结构**两方面考虑。

聚类法和倒排索引 IndexIVFFlat

聚类法是将文本的向量分组聚类，相当于**先进行预训练把相似的向量放到一起**，核心步骤如下：

聚类训练：用 K-means 算法将**全体向量**划分为 **nlist** 个簇，得到簇中心点。这里的簇中心点是先随机选择一个向量，然后计算划到该簇的向量的**平均值作为质心**，不断迭代直至收敛；

向量分配：将每个向量分配到距离最近的簇中心对应的簇中，每个簇以列表的形式存储划分过来的向量。

倒排索引：建立**簇 ID**（非簇中心向量）→ **向量列表**的映射关系，形成**倒排索引表**。

```
# 训练聚类器 (K-means 过程)
assert not index.is_trained
index.train(data)
# 输出：训练耗时和收敛情况
assert index.is_trained
```

这时我们再看一下**向量和索引是如何变化的**？

向量分配到各个簇后，此时**物理存储结构会变**，但是**逻辑 id 不变**，搜索时返回的也是这个**逻辑 id**。

索引结构存储了 `nlist` 个簇中心向量，每个**中心向量对应一个列表**，里面存储该簇的向量（可以**类比为字典**，簇 **id 为键**，簇**向量列表为值**）

检索优化：搜索时先计算查询向量与所有簇中心向量的距离，然后选出最近的前 **nprobe** 个簇，再从选出的簇向量进行二次计算，大幅减少计算量。

如果在训练后添加新的数据，是否需要重新训练？通常不需要，但如果有大量新数据导致数据分布变化显著，可能需要重新训练或增量更新簇中心，但 Faiss 本身不支持在线更新簇中心，需要**重新训练整个索引**。

上面每个簇存储的还是原始的高维向量，这种基础的方式称为 **IndexIVFFlat**，下面是一个简单的数值实例：

训练聚类（`nlist=2`）

簇中心（训练后得到）：

```
centroids = [ [0.3, 0.4, 0.5, 0.6], # 簇 0 中心（虚拟均值）
               [0.7, 0.8, 0.9, 1.0] # 簇 1 中心（虚拟均值） ]
```

倒排列表（向量分配到最近簇）：

```
inverted_lists = { 0: [向量 0, 向量 2], # 簇 0 包含 ID=0 和 ID=2 这里的 id
                   1: [向量 1] # 簇 1 包含 ID=1 }
```

聚类法和乘积量化 **IndexIVFPQ**

乘积量化索引 PQ 的核心思想是分而治之，

首先是把原始高维（`d` 维）向量**分割成 `m` 段子向量**，

然后分别在每个子向量空间进行独立的**预先聚类训练**，得到 **$k=2^n$** （经验值 8）**个聚类中心**（即 `k` 个和子向量维度 `d/m` 一致的向量），称为**该段的码书**，码书训练好就固定不变了，每个聚类中心 `i` 从 0 开始编号，到 `k-1`，

然后再计算**每个子向量**和这 `k` 个聚类中心的相似度（距离），选择最近的聚类中心，用它在上面训练得到的**码本索引 `i`**（会使用 **bits 编码**）作为替代。

这样便可以得到 `m` 个聚类中心索引来表征原始的高维向量。

下面结合具体的例子，来说明上述思想中名词和数字的具体含义，为了让计

算可行，使用一个非常小的维度。假设：

向量维度 $d = 4$

子量化器数量 $m = 2$

每个子量化器的位数 $\text{nbits} = 8$

第 1 步：训练阶段 - 创建每个子空间的“码书”

在添加任何数据之前，IndexIVFPQ 需要先进行**训练**。训练的的目的是为**每一个子空间**学习一套“密码本”或“字典”，专业术语叫 **码书**。

获取训练数据：从数据集中采样大量向量（例如 100000 个）。

分割向量：将每个训练向量都分成 $m=2$ 段。

独立聚类：将所有向量的第一段 $[u1, v1]$ 收集起来，用一个 K-Means 算法进行聚类。将所有向量的第二段 $[u2, v2]$ 收集起来，用另一个 K-Means 算法进行聚类。

训练结果（假设）：

第一段子空间的码书 Codebook1 有了 256 个聚类中心。我们假设前 3 个中心是： $C1_0 = [0.1, 0.8]$ (索引 0); $C1_1 = [0.3, 0.6]$ (索引 1); $C1_2 = [0.9, 0.0]$ (索引 2)... (一直到索引 255)

第二段子空间的码书 Codebook2 也有 256 个聚类中心。我们假设前 3 个中心是： $C2_0 = [1.0, 0.5]$ (索引 0); $C2_1 = [0.2, 0.2]$ (索引 1); $C2_2 = [0.7, 0.9]$ (索引 2)... (一直到索引 255)

第 2 步：编码阶段 - 一个具体的数字实例

现在，我们要**添加一个真实的向量**并将其编码。

原始向量： $V = [0.32, 0.61, 0.19, 0.23]$

分段： $\text{Seg1} = [0.32, 0.61]$ // 前两个维度； $\text{Seg2} = [0.19, 0.23]$ // 后两个维度

为每一段寻找最近的聚类中心：

对于 $\text{Seg1} = [0.32, 0.61]$ ，我们计算它与 **Codebook1** 中所有中心的距离（比如用 L2 距离），找到距离最近的那个。

到 $C1_0 [0.1, 0.8]$ 的距离： $\sqrt{((0.32-0.1)^2 + (0.61-0.8)^2)} \approx \sqrt{(0.0484 + 0.0361)} \approx 0.29$

到 C1_1 [0.3, 0.6] 的距离: $\sqrt{((0.32-0.3)^2 + (0.61-0.6)^2)} \approx \sqrt{(0.0004 + 0.0001)} \approx 0.022$

到 C1_2 [0.9, 0.0] 的距离: $\sqrt{((0.32-0.9)^2 + (0.61-0.0)^2)} \approx \sqrt{(0.3364 + 0.3721)} \approx 0.84$

... (其他中心距离更远)

结论: Seg1 离 C1_1 (索引 1) 最近。

对于 Seg2 = [0.19, 0.23], 我们计算它与 Codebook2 中所有中心的距离。

到 C2_0 [1.0, 0.5] 的距离: 很大

到 C2_1 [0.2, 0.2] 的距离: $\sqrt{((0.19-0.2)^2 + (0.23-0.2)^2)} \approx \sqrt{(0.0001 + 0.0009)} \approx 0.032$

到 C2_2 [0.7, 0.9] 的距离: 很大

结论: Seg2 离 C2_1 (索引 1) 最近。

生成编码:

原始向量 V 的最终编码就是这两个索引值: (1, 1)

在存储时, 这两个索引被紧凑地打包为**两个字节**:

00000001 (数字 1 的 8 位二进制表示)00000001 (数字 1 的 8 位二进制表示)

这就是 PQ 量化的奇迹所在: 一个原始的 **4 维浮点数向量** (占 $4 * 4 = 16$ 字节) 被压缩成了仅仅 **2 个字节**! 压缩比为 8:1。对于更真实的维度 (如 d=128), 节省的内存是巨大的。

第 3 步: 搜索时如何计算距离?

搜索的时候同样将查询向量分段, 并计算与聚类中心的距离, 并记录到查询表中, table[i], table[j], 一个查询表有 2^k 值, 计算查询向量与数据库中存储的任一向量仅需查看原始向量每个子段的索引即可, 并将 m 个子段值加起来。

当查询向量 $Q = [q1, q2, q3, q4]$ 来时, 如何计算它与库中所有压缩向量的距离?

预处理: 将查询向量 Q 也分成两段: $Q1 = [q1, q2], Q2 = [q3, q4]$ 。

预先计算距离表: 这是一个关键优化!

计算 Q1 到 Codebook1 中 **所有** 256 个中心的距离, 得到一个大小为 256 的**查找表 Table1**。

Table1[0] = distance(Q1, C1_0); Table1[1] = distance(Q1, C1_1); Table1[2] = distance(Q1, C1_2); ...

计算 Q2 到 Codebook2 中 所有 256 个中心的距离，得到另一个查找表 Table2。

快速计算近似距离：对于数据库中每个被编码为 (i, j) 的向量（例如我们编码的 $(1, 1)$ ），其与查询向量 Q 的近似距离可以通过查表快速得到：

$$\text{Approximate_Distance} = \text{Table1}[i] + \text{Table2}[j]$$

对于我们的例子：Distance $\approx$ Table1[1] + Table2[1]

通过这种查表法，计算一个距离只需要 **m 次加法操作**，速度极快。

这种组合使得 IndexIVFPQ 在十亿级别的数据集上也能实现毫秒级的搜索，同时将内存占用降低到原始数据的百分之几。

Faiss 索引首先是一个**搜索加速器**，其次才是一个数据存储器。它的主要目标是快速找到相似项，而**不是完美地保存原始数据**。

m 段是经验值吗？有什么特别含义？

m 不是一个纯经验值，它是一个需要根据原始维度 d 和目标来调整的关键超参数。

数学约束：d 必须能被 m 整除。每个子向量的维度是 d/m。

精度 vs. 速度/内存的权衡：

m **越小**（例如 m=1）：子向量维度 d/m 很大。每个子空间需要非常多的聚类中心（比如 65536 个）才能较好地覆盖高维空间，否则量化误差会很大。**结果：**精度可能较低，码书会很大。

m **越大**（例如 m=d）：子向量维度 d/m=1，变成了标量量化。每个子空间只需要很少的中心（比如 256 个）就能很好地覆盖一维直线。**结果：**压缩率最高，内存最小，但**不同维度之间的相关性被完全忽略**，可能会损失精度。

实践中的选择：通常，m 被设置为 d 的一个分数，使得**每个子向量的维度是 2 到 8 之间的一个数**。例如，对于 d=128 的向量，一个常见的选择是 m=16 或 m=32。这样每个子向量是 8 维或 4 维。

n bits=8 的特别含义是什么？

nbits 决定了**每个子量化器有多少个聚类中心**。计算公式是：k = 2^n bits。所以 nbits=8 意味着**每个子空间会有 2^8 = 256 个聚类中心**。**为什么是 8？** 这是一个经典的权衡。**256 个中心已经能提供一个较好的近似精度**，而最终的编码

恰好是 1 个字节（8 bits），存储效率极高。你也可以选择 nbits=12（4096 个中心，精度更高，但编码用 1.5 字节），但 8 是一个在精度和效率之间非常好的默认平衡点。

HNSW 索引（分层导航小世界图）

核心思想：构建多层图结构，高层为长链接（快速定位），底层为短链接（精细搜索），通过贪婪路由逐步逼近目标 9。

数据结构：

多层图：每层节点代表向量，边表示相似关系。高层稀疏，底层密集。

入口点：顶层随机选择入口节点，逐步向下层搜索

图结构构建（M=2）

分层连接（示例）：

层 2（顶层）：向量 2 → 向量 0（长连接）

层 1（中层）：向量 0 ↔ 向量 2

层 0（底层）：向量 0 ↔ 向量 1 ↔ 向量 2（密集连接）

节点连接关系：

{ 0: {层 0: [1, 2], 层 1: [2]}, 1: {层 0: [0, 2]}, 2: {层 0: [0, 1], 层 1: [0], 层 2: [0]} }

搜索过程

从顶层入口点 向量 2 开始，向下层逐步遍历邻居。

索引类型与特点

索引类型	本质与主要内容	特点与适用场景
faiss.IndexFlatL2(d)	存储所有原始的向量数据（float32 数组）。	精确搜索，速度慢，内存占用大。适合小规模数据集或作为其他索引的量化器 1。
faiss.IndexIVFFlat(quantizer, d, nlist)	1. 聚类中心 (Centroids)：通过训练得到的 nlist 个聚类中心向量。 2. 倒排列表 (Inverted Lists)：每个聚类中心对应一个列表，存储属于该簇的原始向量及其在原始数据集中的 ID16。	近似搜索 1。搜索时先找最近的聚类中心，再在其倒排列表内搜索。需训练，在速度、精度和内存之间平衡。
faiss.IndexHNSWFlat(d, M)	1. 聚类中心：同 IVFFlat。 2. 量化后的向量：原始向量被分解为 m 个子向量，每个子空间用一套码书 (Codebook) 量化编码（如用 8bits 表示）。不存储原始向量。	近似搜索。大幅减少内存占用，但会损失精度。适合超大规模数据集。
faiss.IndexPQ(d, M, nbits)	一种分层图结构（Hierarchical Navigable Small World）。每一层都是一个小小世界图，底层包含所有节	高性能，高召回率，无需训练，但内存消耗较大。适合对速度和精度要求高的场

索引类型	本质与主要内容	特点与适用场景
	点，上层是下层的“快车道”。节点是向量，边代表向量间的相似关系。	景。

为什么需要获取向量的维度 (d)?

计算距离：所有相似度度量（L2、内积、余弦等）的计算都依赖于维度。两个向量必须维度相同才能计算它们之间的距离。Faiss 需要知道 `d` 来正确执行这些计算。

内存分配与管理：Faiss 需要在内存中为大量向量分配空间。知道 `d`，它才能准确计算需要多少内存。例如，一个有 100 万个 256 维向量的索引，需要 $1,000,000 * 256 * 4 \text{ bytes} \approx 1.024 \text{ GB}$ 的连续内存（假设 float32，4 字节）。

验证输入数据：当你调用 `index.add(vectors)` 时，Faiss 会检查 `vectors.shape[1]` 是否等于索引初始化时指定的 `d`。这是一种防止错误的保护措施。

构建索引结构：对于 IVF 或 PQ 等复杂索引，聚类中心的数量、量化器的子空间数量等参数都直接与原始向量的维度 `d` 相关。

原始文本和索引的关系

我们已经知道，索引只负责存储向量保存向量的逻辑 id，即默认的隐式 ID 分配模式。Faiss 自动分配从 0 开始的连续整数，按 add 顺序分配。ID 直接对应 向量被添加到索引中的顺序。它完全不知道这些向量代表什么文本、图片或其他内容，检索时返回的也是向量的逻辑 id，因此，需要我们手动建立并维护 **向量逻辑 ID 与原始文本** 的对应关系。

概念辨析

向量：一个句子一个向量表示，多个句子组成二维数组

逻辑 ID：最原始就是向量的行数（从 0 开始计数），可以映射为自定义 ID（和原始 ID 数组对应）

索引结构：向量的组织形式，普通数组，倒排，量化，图索引

langchain 下的 ID：本质是将自定义 ID（整数）改为 uuid，仍和原始位置 id 对应，同时建立了 uuid 和文本的字典对应

自定义 id 与 IndexIDMap

faiss 提供了 IndexIDMap 来支持自定义逻辑 ID（必须是 int64 类型）。这时，数据库主键可能不是简单的 0, 1, 2，而是其他 ID 系统（例如 UUID 或数据库自增 ID）。

基础索引的内部 ID：基础索引（如 IndexFlatL2）存储向量时，会为每个向量分配一个默认的整数 ID，即向量在索引中的位置索引，从 0 开始累加（0, 1, 2, ...），这是索引内部自动管理的，用户无法修改。

自定义 ID：用户可以通过 IndexIDMap 为每个向量指定一个自定义 ID（必须是 64 位整数），这个 ID 由用户定义（如业务系统中的唯一标识、哈希值等）。

映射关系：IndexIDMap 会维护一个内部 ID → 自定义 ID 的映射表，当用户通过自定义 ID 操作向量（如删除、查询）时，IndexIDMap 会先将自定义 ID 转换为基础索引的内部 ID，再调用基础索引的方法执行操作。

```
import faiss
import numpy as np

d = 768
vectors = np.array([...]) # 向量数据

# 假设数据库主键是 [10001, 10002, 10003]
custom_ids = np.array([10001, 10002, 10003]).astype('int64')

# 1. 创建一个基础索引
base_index = faiss.IndexFlatL2(d)
# 2. 用基础索引包裹成一个支持自定义 ID 的索引
index = faiss.IndexIDMap(base_index)

# 3. 添加向量时，同时指定自定义 ID
index.add_with_ids(vectors, custom_ids)

# 现在，搜索返回的将是自定义 ID [10001, 10002, 10003] 等
distances, indices = index.search(query_vector, k=2)
print("返回的自定义逻辑 ID:", indices[0])
```

文档的增删改与索引变化

FAISS 不会自动分开索引库，一个 index 就是一个库，所有向量都存在一起，要分开必须手动创建多个 index。

对于创建完成的基本索引，如果后续增加，删除，修改文档的话，索引会如何变化呢？

增加 (Add): 索引是按照从 0 开始的顺序编码向量的，因此可以一直累加应对文档的增加，所有索引都支持，但性能和维护方式不同。

```
# 向已存在的索引添加新文档
# 示例文本库 2
corpus2: List[str] = [
    "The Apollo is a magic human in old ORLAIC myth.",
    "Deep learning is useful in our daily life.",
    "Python is easier to learn than c++.",
    "The Mars is a beautiful planet."
]
# 按照顺序定义 id 和文本的对应关系，注意文本对应的 id 也要增加
start_id=len(seq_id2text)
for i,text in enumerate(corpus2):
    seq_id2text[start_id+i]=text
```

对于聚类的索引如 IndexIVFFlat，它先通过聚类将空间划分成 nlist 个单元（簇），每个向量被分配到最近的簇中。**增加文档后**，为新向量计算其最近的簇中心，然后将它添加到对应簇的倒排列表中，索引大小增加。**大量增加文档后需要重新训练。**

删除 (Remove): **基础索引**一般是不能修改的，因为当删除一个文档之后，向量的逻辑 id 就不是连续的了，会导致搜索返回的 id 对应不上实际的文本。因此**基础索引不支持删除**。只有使用了 IndexIDMap 或 IndexIDMap2 包装的索引才能有效地按 ID 删除。

```
index.remove_ids(np.array([1001]).astype('int64')) # 关键：按 ID 删除

print("删除后 ID 列表:", index.id_map.array) # [10, 20, 50, 40] (顺序可能变,
30 被移除)
```

它并不真的从**基础索引的物理存储中移除数据**（这很昂贵），而是：在**反向映射表中将对应 ID 标记为已删除**。后续搜索时，**跳过**那些被标记的向量。所以，删除后**索引的物理大小可能没变，但 ntotal 会减少**，有效数据量减少了

修改 (Update): FAISS 没有直接的“修改”操作。你必须先“删除”旧的向量，再“添加”新的向量。这本质上是“删”+“增”的组合。

```
# 假设要修改 ID 为 20 的向量
```

```
new_vector_for_20 = np.random.random((1, dimension)).astype('float32')
```

```
# 先删除旧的
```

```
index.remove_ids(np.array([20]).astype('int64'))
```

#再添加新的（使用相同的 ID）

```
index.add_with_ids(new_vector_for_20, np.array([20]).astype('int64'))
```

对于需要频繁增删改的场景，最佳实践是：

使用 IndexIDMap2 包装基础索引（如 IndexFlatL2 或 IndexIVF...）。

通过 add_with_ids 和 remove_ids 方法来管理向量。

对于 IVF 索引，要有定期重新训练的心理准备和流程设计。

如果数据变动非常剧烈，另一种策略是**定期批量重建整个索引**，这有时比频繁增删改更高效。

Faiss 数据的类型和变化流动

以文本为例：

分割后的文本块——向量数组——索引——文本 id 对应词典

FAISS 的 search 方法支持**批量查询**（一次传入多个查询向量），因此返回的 distances 和 indices 是**二维数组**：

- 第一维对应查询向量的索引（如 distances[0]表示**第 1 个查询向量**的结果）；
- 第二维对应每个查询返回的 k 个结果（按相似度排序，第 0 个最相似）。

例如，当 k=2 且只有 1 个查询向量时：

- distances 的形状为(1, 2) → distances[0]是长度为 2 的列表（存储 2 个距离值）；
- indices 的形状为(1, 2) → indices[0]是长度为 2 的列表（存储 2 个向量索引）。

基于 LangChain 集成的 FAISS

LangChain 的 FAISS 封装是“**文档驱动**”的，通过集成**嵌入向量生成、文档管理、索引操作**，大幅降低了构建基于向量的文档检索系统的门槛，适合快速开发 LLM 应用。而原生 FAISS 是“**向量驱动**”的，专注于高效的向量索引与计算，适合需要深度定制向量处理逻辑的场景（如大规模向量优化、自定义索引策略等）。

也就是说，langchain 框架主要是**处理文档**，而 faiss 只是**调用的一个工具**，这也就意味着 faiss 对向量索引的优化在 langchain 框架下不能很好地使用。

LangChain 提供了一层优雅的抽象,专门用于处理向量存储中“向量”、“ID”和“原始文本(及元数据)”之间繁琐但至关重要的映射关系。其核心设计哲学是:开发者只需关心文档(Document)本身,而无需手动维护 ID 映射。LangChain 会自动处理这些细节。

LangChain 通过以下两个核心组件协同工作,建立并维护 ID 与文本的映射:

向量存储 (Vectorstore): 负责存储向量数据以及向量与 ID 的映射(例如通过 IndexIDMap),并执行相似性搜索。它返回的是与查询最相似的向量的 ID。

文档存储 (Docstore): 负责存储 ID 到完整 Document 对象的映射。Document 对象包含了 page_content (原始文本) 和 metadata (元数据)。

核心概念 Vectorstore (向量存储):

定义: 一个用于存储、检索和管理向量嵌入(embeddings)的抽象类层。

在 LangChain 中的角色: 它是一个接口,定义了所有向量数据库(如 FAISS, Chroma, Pinecone)都需要实现的一系列标准方法(如 add_documents, similarity_search)。

FAISS 类是 LangChain 对原生 FAISS 库的一个包装器,实现了 Vectorstore 接口。其通过属性和方法重新实现了原生 faiss 的索引结构,索引添加向量,向量 id 和文本的对应关系。

Vectorstore 创建方式与基本属性方法

基于 FAISS 的 Vectorstore 创建

最常用的方法是从文档直接创建,LangChain 会自动处理所有底层细节(文本嵌入、索引创建、ID 生成、映射维护)。

```
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
# 准备文档
documents = [
    Document(page_content="LangChain is a framework for LLM applications.",
             metadata={"source": "doc1"}),]
embeddings = OpenAIEmbeddings()
# 创建 Vectorstore - 最常用的方式
vectorstore = FAISS.from_documents(documents, embeddings)
# 也可以从已存在的索引和文档存储加载#
vectorstore = FAISS(embedding_function=embeddings, index=loaded_index,
                    docstore=loaded_docstore, index_to_docstore_id=loaded_mapping)
```

```

class FAISS(VectorStore):
    def __init__(
        self,
        embedding_function: Callable,
        index: Any,
        docstore: Docstore,
        index_to_docstore_id: Dict[int, str],
        normalize_L2: bool = False,
    ):
        # 存储核心组件
        self.embedding_function = embedding_function
        self.index = index
        self.docstore = docstore
        self.index_to_docstore_id = index_to_docstore_id

```

基本属性:

`embedding_function`: 用于生成向量嵌入的模型。

`index`: 底层的 FAISS 索引对象（如 `IndexFlatL2`, `IndexIDMap`）。

`docstore`: 与之关联的文档存储对象。

`index_to_docstore_id`: 一个字典，映射 **FAISS 索引内部 ID** 到 **Docstore 中的文档 ID**。

核心方法:

`from_documents(documents, embedding, **kwargs)`: 类方法。创建索引、生成嵌入、添加文档、建立所有映射。

`from_texts(texts, embedding, metadatas=None, **kwargs)`: 类方法。与 `from_documents` 类似，但直接接收文本列表。

`add_documents(documents, **kwargs)`: 实例方法。向已存在的索引中添加新文档。

`similarity_search(query, k=4, **kwargs)`: 实例方法。执行相似性搜索，返回一个 `Document` 对象的列表。

`similarity_search_with_score(query, k=4, **kwargs)`: 实例方法。返回文档及其相似度分数。

`save_local(folder_path, index_name="index")`: 将索引和文档存储保存到磁盘。

`load_local(folder_path, embeddings, index_name="index", **kwargs):` 类方法。
从磁盘加载。

FAISS.from_documents

1. FAISS.from_documents 内部原理

步骤 1: **文本分割** (如果必要)

虽然传入的是一个 Document 列表, 但每个 Document 可能包含较长的文本。
LangChain 的文本分割器 (如果指定) 会将每个文档分割成更小的块 (chunks)。

步骤 2: **向量化**

使用传入的 `embedding_model` (一个 Embedding 模型适配器) 将每个文本块转化为向量。这个适配器会调用 **底层的嵌入模型** (例如通义千问的嵌入模型) 来生成向量。

步骤 3: **构建 Faiss 索引 index**

根据指定的参数 (如 `normalize_L2` 和 `distance_strategy`) 创建 Faiss 索引 index。

- `normalize_L2=True`: 表示在添加向量之前会对向量进行 L2 归一化。归一化后的向量点积等于余弦相似度。

- `distance_strategy="METRIC_INNER_PRODUCT"`: 设置 Faiss 使用内积作为相似度度量。因为归一化后的内积等于余弦相似度, 所以这样设置是为了进行余弦相似度搜索。

步骤 4: **添加向量和元数据**

- 将向量添加到 Faiss 索引中。

- 同时, 将每个文本块的元数据 (来自 Document 的 `metadata`) 存储在一个 **独立的结构中** (通常是列表或字典), 以便在搜索时能够返回对应的元数据。

步骤 5: **返回 FAISS 向量存储对象**

- **返回一个 FAISS 对象, 该对象包含:**
 - Faiss 索引 (用于向量搜索)
 - 存储的元数据 (用于返回搜索结果时的额外信息)
 - 嵌入模型 (用于在添加新文档时进行向量化)

@classmethod

def from_documents(

`cls, documents: List[Document], embedding: Embeddings, **kwargs,) -> FAISS:`

1. 提取文本内容

`texts = [doc.page_content for doc in documents]`

2. 调用 from_texts 核心方法处理文本

`return cls.from_texts(texts, embedding, metadatas=[doc.metadata for doc in documents], **kwargs)`

@classmethod

def from_texts(

`cls, texts: List[str], embedding: Embeddings, metadatas: Optional[List[dict]] = None, **kwargs,) -> FAISS:`

3. 生成嵌入向量

`embeddings = embedding.embed_documents(texts)`

4. 创建 Faiss 索引

```
dim = len(embeddings[0])
index = faiss.IndexFlatIP(dim)
```

5. 归一化处理

```
if kwargs.get("normalize_L2", False):
    faiss.normalize_L2(np.array(embeddings))
index = faiss.IndexIDMap(index)
```

6. 添加向量到索引

```
index.add(np.array(embeddings, dtype=np.float32))
```

7. 构建文档存储

```
docstore = InMemoryDocstore()
index_to_id = {}
for idx, text in enumerate(texts): # idx 是文本块的索引顺序，也是内部 id
    metadata = metadatas[idx] if metadatas else {}
    doc_id = str(uuid.uuid4()) # 使用 uuid 方法生成文档 id
    docstore.add({doc_id: Document(page_content=text, metadata=metadata)})
    index_to_id[idx] = doc_id
```

事实上索引内部的 id 并没有改变，只是额外增加了一层映射，将内部 id 和自定义的 id 如 doc-id 绑定起来，然后再利用 doc-id 和文本建立的映射关系返回对应文本。

8. 返回向量存储对象

```
return cls(embedding, index, docstore, index_to_id, **kwargs)
```

从上面的代码可以看出来，langchain 下的 from_documents 只是封装了单独使用 faiss 的步骤，其基本原理是一致的。

add_embeddings 方法

原理：将文本向量和元数据添加到 FAISS 索引和文档存储器中。

```
# 源码路径：langchain/vectorstores/faiss.py
def add_embeddings(self, text_embeddings: List[Tuple[str, List[float]]],
                  metadatas: Optional[List[dict]] = None) -> None:
    embeddings = np.array([emb for _, emb in text_embeddings], dtype=np.float32)
    self.index.add(embeddings) # FAISS 原生添加操作
```

LangChain 扩展：存储文档内容

```
texts = [text for text, _ in text_embeddings]
for idx, text in enumerate(texts):
    metadata = metadatas[idx] if metadatas else {}
    doc = Document(page_content=text, metadata=metadata)
    doc_id = str(uuid.uuid4())
```

```
self.docstore.add({doc_id: doc})
self.index_to_docstore_id[idx] = doc_id
```

save_local 方法

原理：保存索引到磁盘（包含 FAISS 索引 + 文档元数据）。

def save_local(self, folder_path: str) -> None:

```
# 保存 FAISS 索引
faiss.write_index(self.index, f'{folder_path}/index.faiss")
# 保存 LangChain 特有数据
with open(f'{folder_path}/index.pkl", "wb") as f:
    pickle.dump((self.docstore, self.index_to_docstore_id), f)
```

load_local 方法

原理：从磁盘加载索引及关联文档数据。

@classmethod

def load_local(cls, folder_path: str, embeddings: Embeddings) -> FAISS:

```
# 加载 FAISS 索引
index = faiss.read_index(f'{folder_path}/index.faiss")
# 加载 LangChain 特有数据
with open(f'{folder_path}/index.pkl", "rb") as f:
    docstore, index_to_docstore_id = pickle.load(f)
return cls(embeddings, index, docstore, index_to_docstore_id)
```

similarity_search

搜索请求入口：用户调用 vectorstore.similarity_search("what is FAISS?")。

查询向量化：vectorstore 使用其 embedding_function 将查询文本转换为一个向量 query_vector。

向量相似性搜索：vectorstore 调用 self.index.search(query_vector, k=4)。
底层的 FAISS 索引返回一个包含内部 ID 和距离的数组，例如 ([0, 2, 5], [0.1, 0.2, 0.3])。

ID 转换与文档检索：

vectorstore 查看 self.index_to_docstore_id 映射字典，将内部 ID [0, 2, 5] 转换为 Docstore 中的 ID ['uuid-123', 'uuid-456', 'uuid-789']。

然后，它调用 self.docstore.search('uuid-123')，
self.docstore.search('uuid-456')...（实际上是一次性批量获取）。

返回结果: docstore 返回对应的 Document 对象。vectorstore 将这些对象收集到一个列表中并返回给用户。用户最终得到的是包含原始文本和元数据的文档，完全无需感知底层复杂的 ID 映射和向量处理。

Docstore (文档存储)探究

在原生的 faiss 实现中，我们需要手动维护一个向量 id 到原始文本的映射，而且原始文本和向量是独立存在的，而 langchain 框架则使用 vectorstore 的一个属性（本质是字典）来统一处理存储原始文本和元数据，这就是 docstore。

在 LangChain 中的角色: 它是一个简单的**存储抽象**，通常**基于字典实现**（如 InMemoryDocstore），负责维护 ID 到 Document 对象（包含文本和元数据）的字典。

通常你不需要直接创建它。它在 FAISS.from_documents() 过程中被自动创建和关联。在底层，它通常是一个 InMemoryDocstore 实例。

本质上，LangChain 内部是这样做的：

```
from langchain_community.docstore.in_memory import InMemoryDocstore
```

```
docstore = InMemoryDocstore({
    "uuid-1234": Document(page_content="...", metadata={...}),
    "uuid-5678": Document(page_content="...", metadata={...}),})
```

在 LangChain 中，向量 ID、索引（向量存储的位置）与文本（原始文档）的映射关系是通过**三层结构**实现的，核心目的是确保“**向量检索结果**”能准确关联到“**原始文档内容**”。

这三层结构分别是：**FAISS 向量索引**（内部向量 id）、**index_to_docstore_id 映射表**（内部向量 id→自定义文档 ID）、**DocStore**（自定义文档 ID→原始文本）。

- 每个向量在 FAISS 索引中都有一个唯一的**位置索引**（int 类型，如 0、1、2...代表第 0 个向量、第 1 个向量）；
- 这个位置索引通过 index_to_docstore_id（字典，key 为位置，value 为文档 ID）映射到唯一的自定义**文档 ID**（如 "doc-1"、"uuid-xxx"）；
- 文档 ID 再通过 docstore 映射到包含原始文本的 **Document 对象**（字典，key 为文档 ID，value 为 Document 对象，包含原始文本）。

LangChain 的 FAISS VectorStore 的 index_to_docstore_id 字典映射和 indexIDMap 的核心差异如下：

实现方式	原生 FAISS 的 indexIDMap	LangChain 的 index_to_docstore_id
ID 类型	仅支持 64 位整数（int64）	支持任意哈希 able 类型（如字符串、UUID）
映射方向	向量→自定义 ID（通过 add_with_ids 关联）	向量位置索引→文档 ID（字典映射）

实现方式	原生 FAISS 的 <code>indexIDMap</code>	LangChain 的 <code>index_to_docstore_id</code>
依赖对象	包装在 FAISS 索引内部，与索引绑定	独立于 FAISS 索引的 Python 字典
用途	向量与自定义 ID 的直接关联，支持 ID-based 操作	向量位置与文档 ID 的关联，用于对接 DocStore
动态更新	支持通过 <code>remove_ids</code> 删除向量（基于自定义 ID）	删除向量需同步更新字典，操作更繁琐

文档的增删改

LangChain 中文档的本质是 Document 对象（包含 `page_content` 文本内容、`metadata` 元数据和 `id` 唯一标识）。操作流程如下：

- **新增**：生成文档向量 → 存入 VectorStore → 原始文档存入 DocStore → **更新映射关系**；
- **删除**：从 VectorStore 中删除对应向量 → 从 DocStore 中删除原始文档 → **更新映射关系**；
- **修改**：无直接修改接口，需通过“删除旧文档 + 新增修改后文档”实现（因向量由文本生成，文本修改后向量必变）。

文档新增（Add）

新增文档是最基础的操作，LangChain 的 VectorStore 提供了 `add_documents` 方法，自动完成向量生成、存储及映射同步。

```
from langchain.vectorstores import FAISS
from langchain.embeddings import OpenAIEmbeddings
from langchain.docstore.document import Document
# 初始化嵌入模型和空的 FAISS 向量库
embeddings = OpenAIEmbeddings()
# 创建空向量库（需手动初始化 DocStore 和映射，实际可直接用 from_documents 创建）
vs = FAISS.from_documents([], embeddings)
# 1. 准备新增文档（可指定 ID，也可自动生成）
new_docs = [
    Document(
        page_content="LangChain 支持多种向量存储后端，如 FAISS",
        metadata={"source": "doc1"},
        id="doc-1" # 自定义文档 ID
    ),
    Document(
        page_content="RAG 是检索增强生成的缩写，核心是结合检索和 LLM 生成",
```

```

        metadata={"source": "doc2"},
        id="doc-2"
    ])
# 2. 新增文档（自动处理向量生成、存储和映射）
vs.add_documents(new_docs)
# 3. 验证新增结果
print(f'向量库总数: {db.index.ntotal}') # 输出: 2（新增 2 个向量）
print(f'DocStore 中的文档 ID: {list(vs.docstore.docs.keys())}') # 输出: ['doc-1', 'doc-2']
print(f'向量索引→文档 ID 映射: {db.index_to_docstore_id}') # 输出: {0: 'doc-1', 1: 'doc-2'}
# 4. 检索验证（能找到新增文档）
query = "什么是 RAG? "
results = db.similarity_search(query, k=1)
print(results[0].page_content) # 输出: "RAG 是检索增强生成的缩写..."
内部实现逻辑
add_documents 方法的核心步骤（简化）：

```

1. 对输入的 Document 对象，若未指定 id 则自动生成（如 UUID）；
2. 调用嵌入模型（embedding_function）将 page_content 转换为向量；
3. 将向量添加到 FAISS 索引（index.add(vectors)），此时向量在索引中的位置为[当前 ntotal, 当前 ntotal+1, ...]；
4. 将文档存入 DocStore（docstore.add(docs)）；
5. 更新 index_to_docstore_id 映射：{新向量位置: 文档 ID, ...}。

文档删除（Delete）

LangChain 未直接提供 delete_documents 方法，需通过“查找文档 ID 对应的向量索引→删除向量→删除 DocStore 文档→更新映射”手动实现。

```

# 承接上文的 vs 对象（已包含 doc-1 和 doc-2）
# 1. 确定要删除的文档 ID
doc_id_to_delete = "doc-1"
# 2. 查找文档 ID 对应的向量索引（从 index_to_docstore_id 中反向查找）
index_to_delete = None
for idx, doc_id in vs.index_to_docstore_id.items():
    if doc_id == doc_id_to_delete:
        index_to_delete = idx
        Break
if index_to_delete is None:
    raise ValueError(f'文档 ID {doc_id_to_delete} 不存在')
# 3. 从 FAISS 索引中删除向量（需先转换为索引列表）# 注意：FAISS 的
remove_ids 接收的是内部索引（0,1,2...），而非自定义 ID
vs.index.remove_ids(np.array([index_to_delete], dtype=np.int64))
# 4. 从 DocStore 中删除文档

```

```

vs.docstore.delete([doc_id_to_delete])
# 5. 更新 index_to_docstore_id 映射（删除对应键值对，并重新对齐后续索引）#
原映射：{0: 'doc-1', 1: 'doc-2'} → 删除 0 后，需将 1 的索引改为 0
new_index_to_docstore_id = {}
for idx, doc_id in vs.index_to_docstore_id.items():
    if idx != index_to_delete:
        # 若索引大于被删除的索引，需减 1（因 FAISS 删除后后续向量索引自
        # 动前移）
        new_idx = idx - 1
        if idx > index_to_delete else idx
        new_index_to_docstore_id[new_idx] = doc_id
vs.index_to_docstore_id = new_index_to_docstore_id
# 6. 验证删除结果
print(f'删除后向量库总数: {vs.index.ntotal}') # 输出：1
print(f'DocStore 剩余文档 ID: {list(vs.docstore.docs.keys())}') # 输出：['doc-2']
print(f'更新后映射: {vs.index_to_docstore_id}') # 输出：{0: 'doc-2'}
# 7. 检索验证（无法找到被删除的文档）
query = "LangChain 支持哪些向量存储？"
results = vs.similarity_search(query, k=1)
print(results) # 输出：[]（因 doc-1 已被删除）

```

- FAISS 索引删除向量后，**后续向量的内部索引会自动前移**（如删除索引 0 后，原索引 1 变为新索引 0），因此必须同步更新 index_to_docstore_id 的键（索引值）；
- 若使用 IndexIDMap 包装的 FAISS 索引（支持自定义 ID），删除逻辑可简化（直接通过自定义 ID 删除向量），但 LangChain 的 FAISS 类默认不使用 IndexIDMap，需手动改造。

文档修改（Update）

由于文档向量由文本内容生成，修改文本后向量必然变化，因此 LangChain 中“修改”需通过“删除旧文档 + 新增修改后文档”实现。

```

# 承接上文的 vs 对象（当前包含 doc-2）
# 1. 定义要修改的文档 ID 和新内容
old_doc_id = "doc-2"
new_doc = Document(
    page_content="RAG（检索增强生成）是一种结合信息检索和大语言模型的技术，能提升生成内容的准确性", # 修改后的文本
    metadata={"source": "doc2-updated"}, # 修改元数据
    id=old_doc_id # 保留原 ID（或换新 ID）
)
# 2. 先删除旧文档（复用删除逻辑）# 查找旧文档索引
index_to_delete = None
for idx, doc_id in vs.index_to_docstore_id.items():
    if doc_id == old_doc_id:
        index_to_delete = idx

```



```

        break# 删除向量
vs.index.remove_ids(np.array([index_to_delete], dtype=np.int64))# 删除 DocStore 文档
vs.docstore.delete([old_doc_id])# 更新映射
new_index_map = {idx: doc_id for idx, doc_id in vs.index_to_docstore_id.items() if
idx != index_to_delete}
vs.index_to_docstore_id = new_index_map
# 3. 新增修改后的文档
vs.add_documents([new_doc])
# 4. 验证修改结果
print(f'修改后向量库总数: {vs.index.ntotal}') # 输出: 1 (删除 1 个+新增 1 个)
print(f'DocStore 中的文档内容:
{vs.docstore.search([old_doc_id])[0].page_content}') # 输出修改后的文本
# 5. 检索验证 (匹配修改后的内容)
query = "RAG 的作用是什么? "
results = vs.similarity_search(query, k=1)
print(results[0].page_content) # 输出: "RAG (检索增强生成) 是一种结合..."

```

- 若希望保留原文档 ID，新增时使用原 ID 即可；若需换新 ID，直接指定新 ID 即可；
- 本质是通过“删除旧向量 + 生成新向量 + 新增新向量”实现修改，确保向量与文本内容的一致性。